

NexaFreight Control Tower — Complete Project Report

A logistics control tower with OSINT-driven disruption intelligence

Report generated: 6 September 2026 · Plan version: Scheduled Flow

v2.2 · Companion document:

`nexa_freight_scheduled_flow_v2.html` (the task-level schedule)

Table of Contents

1. [Executive Summary](#)
 2. [What the Project Is](#)
 3. [How We Got Here — Project History](#)
 4. [Where We Are Right Now](#)
 5. [System Architecture](#)
 6. [Technology Stack & Hard Constraints](#)
 7. [Core Design Disciplines](#)
 8. [Data & Machine Learning](#)
 9. [The Intelligence Layer \(OSINT → Logistics\)](#)
 10. [The Policy & Rules Layer](#)
 11. [User Roles & the Application Surface](#)
 12. [Team Structure — the 4-Role Model](#)
 13. [The Roadmap, Phase by Phase](#)
 14. [Milestones & Timeline](#)
 15. [Stretch Backlog \(post-v2.0\)](#)
 16. [Open Decisions](#)
 17. [Risk Register](#)
 18. [What to Expect at the Very End](#)
 19. [Appendix — Standing Conventions, Credentials, Glossary](#)
-
-

1. Executive Summary

NexaFreight is a logistics control tower: a system that tracks multimodal freight shipments (sea, air, road, rail) on a live map, predicts delays with explainable machine learning, watches the world's news and disaster feeds for events that threaten those shipments, quantifies the financial damage, and helps a human operator decide what to do about it — reroute, split the shipment, switch transport mode, or accept the delay — with every decision recorded against the exact policy rules that priced it.

The NexaFreight frontend is a Next.js OSINT globe application — live weather, conflicts, news, entity knowledge graphs, vessel/flight/truck tracking overlays — serving as the platform's presentation layer (built on an open-source OSINT codebase, aggressively pruned and extended for logistics). The non-negotiable architectural rule: **the frontend is an HTTP-only client**. All business logic, all impact analysis, and every number lives in the NexaFreight FastAPI backend.

Status: 39 of the original 72 tasks are done (54%), plus the full 3-phase frontend integration (auth, map, live SSE telemetry). The ML models are trained. What remains is the "intelligence" half of the product: disruption detection, alerting, financial exposure, rerouting decisions, the copilot, analytics, and hardening — **58 core tasks (50 build + 8 research) plus 5 stretch**, executed over **11 weeks, 7 Sep → 22 Nov 2026**, by **3 engineering roles + 1 research member** who owns a **conference paper** about the system.

Budget: \$0. Everything runs on free tiers and keyless public feeds, on developer laptops, with SQLite + DuckDB for storage.

2. What the Project Is

2.1 The problem

Freight shipments are late, a lot. When a crisis hits — a port strike, a

cyclone, a canal closure, a war-risk spike in a strait — logistics teams find out late, price the impact slowly, and decide ad hoc. Commercial control towers (project44, FourKites, Flexport, SAP TM, Kinaxis) solve this for enterprises at enterprise prices, as closed black boxes.

2.2 The product (as defined in the original 40-page plan)

A **control tower** with five capabilities:

1. **Monitor** — every shipment, vessel, truck, and flight live on a 3D globe, with per-port congestion and feed-health indicators.
2. **Predict** — per-shipment delay probability, P10/P50/P85 ETA ranges (never a single date), and per-lane demand forecasts, all explainable (SHAP factor attribution in plain English).
3. **Detect disruptions** — from telemetry (AIS deviation, port congestion) *and* from the news (GDELT conflicts, GDACS disasters, NASA EONET storms), geo-matched to the ports, lanes, and shipments actually affected.
4. **Quantify** — financial exposure per alert: SLA penalties, demurrage at real per-port tariff rates, carbon cost — all computed from a versioned, researched policy layer rather than hardcoded guesses.
5. **Decide** — for each alert, three scored options (e.g. accept delay / divert via Cape of Good Hope / split to air freight), an ☆ AI recommendation, role-gated human approval, map redraw with the new route, and an immutable audit trail.

Plus: an **AI copilot** (Gemini free tier, Ollama offline fallback) that answers "why is this shipment at risk?" citing the actual prediction and the actual news event — and a **conference paper** documenting the whole system.

2.3 What it is not

- Not a rate-shopping or booking platform. It monitors and decides; it does not transact with carriers.
- Not a fully autonomous agent. The design is deliberately **human-in-the-loop** — the AI recommends and prices, humans approve.

High-severity news detections enter as *candidates* requiring one-click operator confirmation.

- Not production SaaS. It is a portfolio/research-grade system with real data, real models, and honest labels, targeting a demo and a paper by 22 Nov 2026.

3. How We Got Here — Project History

The project is the merge of three documented efforts. All three source documents were reviewed in full and every claim was verified against the actual repositories.

Source document	Contents	Role in the project
NexaFreight.pdf (40 pp product plan)	Control tower definition, full stack, DB schema, adapter pattern, provenance rule, Phase 0–7 plan, \$0 budget	Canonical product scope
scheduled_flow.pdf (22 pp, 72 tasks)	Task-by-task chronological plan T-001→T-072 with owners, dependencies, per-phase Definition-of-Done	The numbering system the current schedule continues
Project-state memo (11 pp)	Integration rules, pruning decisions, verified status T-001–T-039, corrections vs plan	Baseline for "where we are"

Plus three repositories, inspected file-by-file:

- **RAHUL4870/nexafreight** — the new integration repo (backend snapshot + globe-frontend app + docs + datasets). The primary codebase going forward.

- **RAHUL4870/Winter_Soldiers** — the old repo; original backend, ends at T-039. Read-only archive after Phase R.
- **The upstream open-source OSINT reference project** — the dashboard the globe UI builds on (~45 API routes; CCTV/satellites/markets layers pruned).

Timeline so far

- **Original plan executed T-001 → T-039** (Phases 0–3: foundation, data foundation, live map, ML models) in the old repo.
 - **Frontend integration Phases 1–3** (auth + client SDK → map routes/ports/inspectors → live SSE telemetry with provenance badges and feed-health HUD) in the new repo.
 - **v2.1 policy layer** (6 Sep 2026): internet research into real industry rules (demurrage tariffs, SLA penalty norms, escalation matrices, routing guides, sanctions screening, GLEC emissions, disruption playbooks) designed into the system as data, not code.
 - **v2.2 research track** (6 Sep 2026): Member 4 added — a non-coding research role owning the conference paper.
-

4. Where We Are Right Now

📍 YOU ARE HERE:

- Original plan: **tasks T-001 → T-039 done** (Phases 0–3), 39/72 = 54%.
- Integration plan: **Phases 1, 2, 3 done** in the new repo.
- Immediate next backend task: **T-040** — `GET /api/shipments/{id}/predict`.
- Immediate programme task: **Phase R** — fix repo drift before building further.

4.1 Verified working (repo-inspected, not assumed)

Backend (FastAPI, async SQLAlchemy + aiosqlite, WAL mode, Python

3.12):

- Package structure, config, DB, 17 ORM models, Alembic migrations 001–002; JWT auth with seeded operator; data pipeline scripts 01–08 (DataCo → orders → shipments → legs, GeoJSON routes with waterway waypoints through Suez/Panama/Malacca, MMSI/flight assignment, AIS parquet).
- Shipment APIs: GET `/api/shipments/{id}/route/events` — paginated, filterable, GeoJSON.
- Feed adapters + workers: AISStream, ReplayAIS, TruckSim, ReplayFlight; ais_listener, position_interpolator (modulo-looping sims so demo assets never freeze).
- Map APIs: SSE stream (`/api/map/positions/stream`, 5 s cadence), snapshot, routes, ports, feed-health.
- **ML models trained + registry + raw predict endpoints:** delay classifier (test ROC-AUC **0.731** vs 0.707 baseline), ETA P10/P50/P85 quantiles, AutoETS demand forecaster (79 of 675 lanes qualified). Trained 4 Sep 2026.

Frontend (Next.js 16 · React 19 · MapLibre 5):

- Login, JWT in-memory auth, `/api/nexa/*` → :8000 proxy, typed client SDK.
- Map: port congestion bubbles, airport badges, sea/air/road/rail polylines, conflict-zone triangles; inspector popups for shipments/ports/airports.
- Live positions over SSE: 36 moving assets (15 vessels · 13 trucks · 8 flights), 4 s glide, heading rotation, route snapping.
- Provenance pills (LIVE / REPLAY / SIM) + feed-health HUD. **431 Vitest tests green, tsc --noEmit 0 errors, pytest green.**

4.2 Verified gaps

- **T-040 is the blocker:** the combined GET `/api/shipments/{id}/predict` endpoint does not exist; only raw feature-payload endpoints (POST `/api/predict/delay`, `/api/predict/eta`) exist, which would force the frontend to assemble ML feature vectors — unacceptable.
- Alerts / disruptions / decisions / analytics / copilot routers are

stubs (8–33 lines each, placeholder dicts).

- Services `disruption_detector`, `alert_engine`, `sla_checker`, `reroute_engine`, `decision_executor`, `copilot` are **not built** (only consolidation, a `financial_engine` skeleton, and `route_planner` exist).
- Kept-but-unwired OSINT layers (weather, conflicts, GDELT, live news, knowledge graph) render as context only — **none are connected to shipments or alerts yet**. Wiring them in is the core of Phase 5.

4.3 Repo drift found (fixed first, in Phase R)

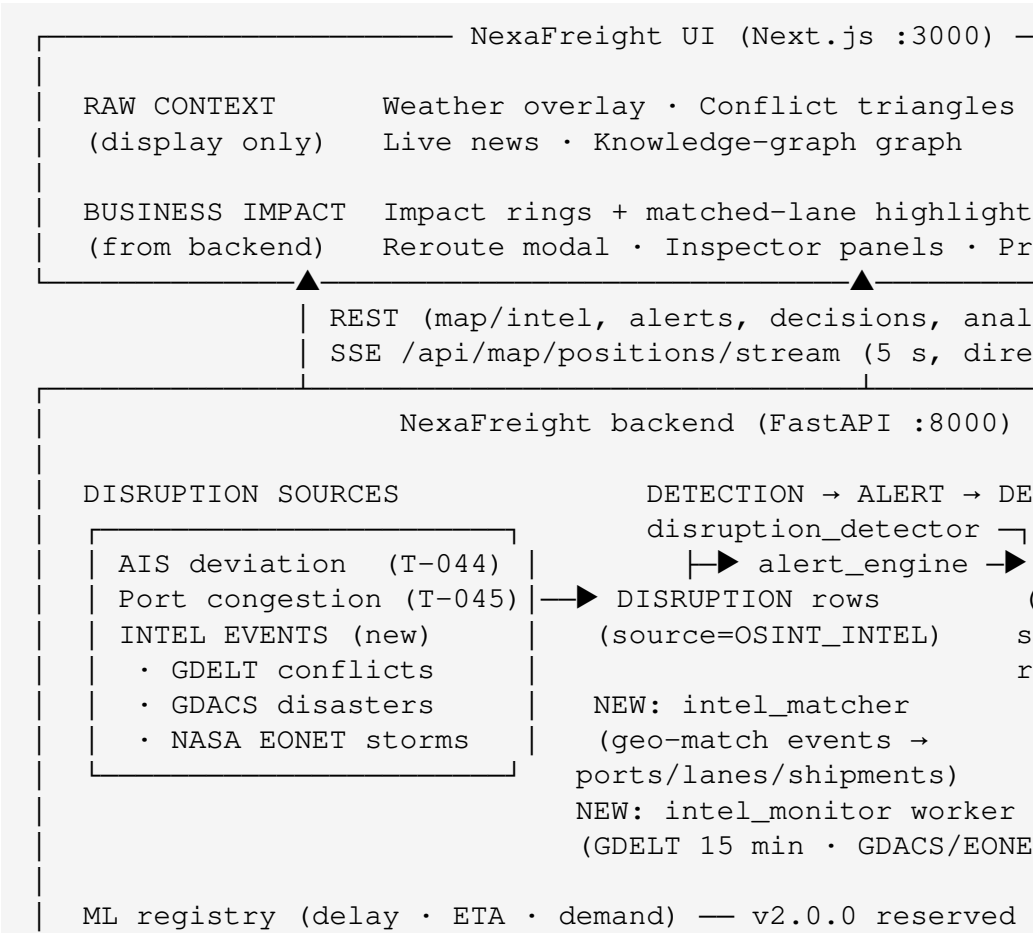
ID	Issue
D1	New repo's embedded backend is <i>behind</i> the old repo HEAD: missing <code>models/event.py</code> , <code>models/reroute_option.py</code> , migrations 003/004, a richer <code>auth.py</code> — while being <i>ahead</i> in waterway routing, looping sims, and the map router. The Alembic chain must be merged before any new migration lands.
D2	Frontend ML client functions (<code>getDelayPrediction/</code> <code>getEtaPrediction/getDemandForecast</code>) were written and type-checked but never pushed — they exist on someone's machine only.
D3	Confusing layout: backend buried at <code>Winter_Soldiers-main/Winter_Soldiers-main/</code> , frontend nested under a stray <code>test/</code> path, ~244 MB of datasets committed in-repo, stray scratch dirs.
D4	Dead UI: layer toggles for removed features (satellites, CCTV, earthquakes, fires), a quakes filter with no quake source, a panel that never renders, an undocumented <code>intel/</code> Express server.
D5	No unified dev experience: no root Makefile, no combined README, CI only covers the old backend, Windows-only runbook paths.

5. System Architecture

5.1 The one rule

The frontend is an HTTP-only client. It never imports Python, never reads the database, never duplicates business logic. Every number it shows comes from a NexaFreight API payload.

5.2 The two processes



news_risk_score, active_disruption_near_dest (stre

Policy engine: every threshold, rate, cap and limit re

policy tables (\$10); every alert/decision records the

5.3 The three-layer rendering rule (keeps the demo honest)

Layer	Contents	Owned by
A — Raw OSINT context	Weather, conflicts, GDELT dots, live news — "what's happening in the world?" No logistics claims.	The frontend's own API routes (free, keyless: GDELT, GDACS, EONET, NWS), unchanged
B — Matched impact	Impact rings, affected-lane highlights, alert cards — "what it means for <i>your</i> freight."	Backend-owned: Python adapters ingest the same public feeds, geo-match to ports/lanes/shipments, materialize <code>intel_events + DISRUPTION(source=OSINT_INTEL)</code> ; The frontend renders strictly from backend payloads
C — Decisions	Alerts → 3 scored options → approval → re-planned legs → new <code>route_version</code> → immutable audit	Backend services + frontend modal

5.4 Request/data flow examples

- **Live map:** browser EventSource → GET /api/map/positions/stream (direct to :8000, not through Next rewrites — proxy buffering breaks SSE) → backend interpolates/simulates/relays AIS positions → markers update every 5 s with provenance pills.
- **News** → **alert:** intel_monitor worker polls GDELT (15 min) / GDACS+EONET (hourly) → events normalized into intel_events → intel_matcher computes affected ports (radius), lanes (polyline ∩ Shapely buffer vs chokepoint registry), and active shipments → DISRUPTION rows → alert_engine recomputes ETAs, runs per-order SLA checks and financial exposure → severity-banded ALERT → operator sees the drawer update.
- **Prediction click:** click a moving asset → inspector panel → one GET /api/shipments/{id}/predict call → feature assembly happens *server-side* → response renders risk badge, P10–P85 bar, top-5 SHAP factors.

5.5 Backend internals

- **APIs:** FastAPI routers — auth, shipments, map, alerts, disruptions, intel, decisions, analytics, copilot.
 - **Services:** route_planner (waterway-aware), financial_engine, sla_checker, disruption_detector, alert_engine, intel_matcher, reroute_engine, decision_executor, policy_engine, copilot, consolidation.
 - **Workers (APScheduler):** ais_listener, position_interpolator, port_monitor (daily 06:00), sla_monitor (15 min), intel_monitor (15 min / 60 min).
 - **Adapters (Protocol pattern):** feeds (AISStream, ReplayAIS, TruckSim, ReplayFlight), intel (gdelt, gdacs, eonet), LLM (gemini, ollama) — every external source behind an interface so tests run against recorded fixtures.
 - **Storage:** SQLite (app state, WAL), DuckDB (analytics, reads SQLite directly), Parquet (AIS history), Alembic migrations, models/ for ML artifacts.
-

6. Technology Stack & Hard Constraints

6.1 Stack

Layer	Choice	Notes
Backend	Python 3.12, FastAPI, async SQLAlchemy, aiosqlite	WAL mode SQLite
Analytics DB	DuckDB	Reads the SQLite file directly; chart-ready aggregation
ML	LightGBM (delay classifier; quantile ETA), statsmodels AutoETS (demand), SHAP (explanations)	Trained artifacts + registry w/ <code>metadata.json</code>
Geo	searoute-py pinned 1.4.3, build-time only ; Shapely buffers/intersection	Routes shipped as GeoJSON data
Frontend	Next.js 16, React 19, MapLibre 5, Zustand-style auth store, <code>recharts</code> for analytics	<code>lightweight-charts</code> stays removed
Live updates	SSE, not WebSocket	One-directional telemetry; direct connect to :8000
LLM	Gemini Flash free tier + Ollama (llama3.1:8b) offline fallback behind one <code>LLMAdapter</code> protocol	429 → Ollama; every call hash-logged to audit
Intel feeds	GDELT GEO 2.0 + DOC, GDACS, NASA EONET, (optional ReliefWeb)	All keyless, all free

Infra	No Docker, no Celery, no Redis, no PostgreSQL. SQLite + DuckDB + APScheduler. Makefiles + GitHub Actions CI	Laptop-runnable, \$0
-------	---	----------------------

6.2 Non-negotiable constraints

- **\$0 budget** — free tiers and keyless public feeds only.
 - **Provenance discipline** — every externally visible value is labeled exactly one of: `REAL` (live external source) · `REPLAYED` (recorded real data, deterministic) · `DERIVED` (computed by us from real inputs) · `CALIBRATED` (industry-informed defaults, source required) · `SIMULATED` (synthetic). Badged visibly in the UI.
 - **Tests are part of done** — backend `pytest` (unit + integration), frontend `Vitest` + `tsc --noEmit`, dual CI from Phase R onward.
 - **Demo determinism** — demos never depend on live-feed roulette; recorded fixtures + `INTEL_AUTO_DISRUPT=false` kill-switch.
-

7. Core Design Disciplines

1. **Provenance everywhere (no dashboard lies).** Five labels, visible badges, stored on every row that leaves the backend. Even intel events follow it: the *event* is `REAL`, its *match* to a shipment is `DERIVED`, policy numbers are `CALIBRATED`.
2. **Policy as data, not code (\$10).** No magic constants in services. Thresholds, rates, caps, and approval limits live in versioned tables resolved through `policy_engine.resolve(key, context)` with `GLOBAL` → customer-segment → port → lane specificity. Every alert and every decision records the `policy_version` used, so six months later you can answer "which rules produced this number?"
3. **Trust gate for news.** News sources generate false positives. Disruptions with estimated severity `HIGH+` enter as `CANDIDATE` and require one-click operator confirmation before financial

exposure is computed; LOW/MEDIUM may auto-create. Gate severities are policy rules, not constants.

4. **Human-in-the-loop decisions.** The AI recommends (☆ pick = best option satisfying SLA under playbook guardrails); a human approves. Approval authority is role-gated by dollar limit (\$11). Executing a decision **replaces** legs, never deletes them; `route_version` increments; the audit log is immutable and idempotent (re-approval → 400).
 5. **Honest UI.** ETAs are always P10–P85 ranges, never a single date. Lanes without demand history render a calm "insufficient history" empty state — never a zeroed chart, never an error toast. Disabled/removed features are pruned, not left as dead toggles.
 6. **Deterministic demos & evaluation.** All recorded fixtures replay exactly; the paper's results tables come from frozen fixtures so numbers never wobble with live feeds.
-

8. Data & Machine Learning

8.1 Datasets

Dataset	Use	Provenance
DataCo Supply Chain (~180k orders)	Training data for delay classification + ETA quantiles; order/shipment generation	REAL (public dataset, usage + license documented in T-100 card)
UN/LOCODE	Port/location reference	REAL
IMF Daily Port Activity + Maritime Port Performance	Port congestion index (today vs 90-day average)	REAL
GDELT GEO 2.0 + DOC	Conflict, political, labour-unrest, keyword themes ("canal closed", "dock workers strike")	REAL (keyless, 15-min cadence)

GDACS	Cyclones, floods, earthquakes, volcanoes with severity	REAL (keyless)
NASA EONET	Active natural events (storms, wildfires)	REAL (keyless)

Pipeline scripts 01–08 turn DataCo into orders → shipments → 600 legs with GeoJSON routes (waterway-aware via Suez/Panama/Malacca waypoints), assign MMSIs/flights, and lay down AIS parquet. The DB file is gitignored — **every machine re-seeds**; datasets (~244 MB) are externalized (release artifact / Drive / Git LFS).

Demo fleet: 36 live map assets — 15 vessels · 13 trucks · 8 flights.

8.2 Models (already trained, artifacts on disk, trained 4 Sep 2026)

Model	Algorithm	Verified performance	Notes
Delay classifier	LightGBM	Test ROC-AUC 0.731 vs 0.707 grouped baseline	Top-5 SHAP factors rendered with direction + plain-English labels
ETA ensemble	LightGBM quantile regression	P10 / P50 / P85 outputs	Contract enforces $P10 \leq P50 \leq P85$ invariant; UI shows ranges only
Demand forecaster	AutoETS per lane	79 of 675 lanes qualified	596 lanes return UNKNOWN_LANE 404 → calm empty state by design

- A model registry validates `metadata.json`; predictions carry `model_version` and `provenance=DERIVED`.
- **A v2.0.0 upgrade slot is already reserved** in the feature schema: `news_risk_score` and

`active_disruption_near_dest`. v1 ignores them safely at inference; activating them is stretch task T-090 (retrain once Phase 5 has logged enough matched intel history; validate AUC lift over 0.731).

8.3 Prediction API surface

- `POST /api/predict/delay`, `POST /api/predict/eta`, `GET /api/demand/forecast` — raw, feature-payload style (exists).
 - `GET /api/shipments/{id}/predict` — **T-040, the current blocker**. Server-side feature assembly → delay probability, `is_high_risk`, top-5 SHAP factors, P10/P50/P85 ETAs, `sla_risk` band, `model_version`, provenance. 503 with a clear message when models are unloaded. After T-040, **no frontend code assembles ML feature vectors anymore**.
-

9. The Intelligence Layer (OSINT → Logistics)

This is the core new scope of v2 — the reason the globe-frontend integration exists. It turns "news is on the map" into "news moves freight."

9.1 Signal → disruption mapping (rule-based v1, no ML needed to start)

Intel signal	Feed	Disruption type	Match rule (v1)
--------------	------	-----------------	-----------------

Armed conflict / strike near port or strait	GDELT GEO+DOC	PORT_STRIKE / ROUTE_RISK	Within R km of port centroid, or route polyline $\cap$ event buffer (Shapely). R: 150 km ports, 400 km chokepoints
Cyclone / typhoon / severe storm	GDACS · EONET	WEATHER	Cone/track buffer $\cap$ sea-leg geometry, or ± 72 h port-call window
Flood / quake / volcano damaging a port region	GDACS · EONET	WEATHER (or PORT_CONGESTION)	Severity $\geq$ Orange, within 100 km of port
"Canal closed" / "strait blockade" / sanctions	GDELT DOC themes	CANAL_CLOSURE / GEOPOLITICAL	Keyword + geofence on the chokepoint registry
Labour unrest at ports	GDELT DOC	PORT_STRIKE	Keyword + port geofence → candidate for operator confirm
Sanctioned vessel (stretch, T-096)	OpenSanctions via intel/ server	COMPLIANCE_HOLD	Vessel IMO $\times$ sanctions list; always manual review

Chokepoint registry (reference data): Bab-el-Mandeb · Hormuz ·

9.2 The other two disruption sources (telemetry)

- **AIS deviation** (T-044): planned-vs-actual progress along a leg > 10% → `DISRUPTION(VESSEL_DELAY, source=AIS_DEVIATION)`, idempotent per vessel+leg.
- **Port congestion** (T-045): daily 06:00 scan; `congestion_index = today / 90-day avg > 1.4` → `DISRUPTION(PORT_CONGESTION, source=PORT_DATA)`.

9.3 From disruption to decision

1. `alert_engine` (T-047): disruption → revised ETAs → per-order SLA check → financial exposure → severity (CRIT > \$20k or > 5 d late · HIGH > \$5k or any breach · MED > \$1k · LOW info). Severity bands and ack-SLAs come from the policy registry. Unacknowledged CRITICAL past its ack-SLA auto-escalates with an audit note. One alert per (shipment, disruption).
 2. `reroute_engine` (T-054): an `ACCEPT_DELAY` baseline option is always present, plus corridor-filtered alternates and SPLIT evaluation, full financials per option, ranked by total impact, `is_ai_recommended` marked, top 3 persisted. Which option types are legal for a disruption type — and the max cost-multiplier / ETA-slip guardrails — come from the `playbooks` table, not engine constants.
 3. `decision_executor` (T-055): approve → old legs REPLACED (not deleted), new legs written, `route_version++`, SPLIT creates a child shipment, immutable `audit_log` entry, P&L recalc, idempotent re-approval guard. Senior-led with pair review — this is the highest-concurrency-risk component.
 4. Map redraws with the new route; replaced legs greyed; origin/dest pinned.
-

10. The Policy & Rules Layer

Added in v2.1 from internet research (6 Sep 2026). The original plan referenced APPLICABLE RULES: `sop_rules` in the copilot context but no task ever created the rules — researching how real freight operators encode policy produced seven rule families with concrete industry numbers, each mapped to a seedable object.

10.1 The seven researched rule families

1. **Late-delivery SLA penalties** — industry norm 0.5–2% of order/freight value per day of delay (3PL norm 1–2%), tiered escalation (1% wk 1 → 2% wk 2), hard caps at 5–15% of order value (most common 10%), grace windows, and force-majeure carve-outs for natural disasters/strikes/government action — exactly what the intel layer detects. *Sources: Ramco, AuraVMS, LawInsider samples.*
2. **Demurrage & detention tariffs** — free time typically 3–7 calendar days import (LA/LB 4–5, NY/NJ 4, Shanghai 3–5, Rotterdam 7, Singapore 5–7); rates are **tiered**, ~\$75–150/day (d1–5) → \$150–250 (d6–10) → \$250–350+ (d11+); ~2× for 40', reefers 2–3×. **FMC OSRA rule (eff. 28 May 2024)**: invoice within 30 calendar days or the charge is unpayable; ≥30-day dispute window; mandatory invoice content. *Sources: Federal Register, ExFreight, FreightAmigo, Suaid.*
3. **Decision rights & escalation** — real control towers publish an authority matrix: pre-approved actions under cost/CO₂ limits (tower executes), recommend-only mid-tier, human-only above spend thresholds; time-based auto-escalation (CRITICAL unacknowledged ~15 min → escalate). Maps onto OPERATOR ≤ \$5k / MANAGER ≤ \$25k / DIRECTOR above. *Sources: FleetRabbit, Umbrex, Hyperping.*
4. **Routing guides & guardrails** — ranked carriers per lane with timed tender cascade (30–45 min response windows; ~2 h for 3+ days out); hazmat/temp/customer restrictions; off-guide routing costs 15–30% more → reroute options need max-cost-multiplier guardrails. *Sources: Owlery, Warematch, USTech.*
5. **Sanctions & compliance screening** — OFAC maritime guidance (Oct 2024 + Apr 2025): screen vessels by **IMO**, not just name; AIS gaps/spoofing are baseline red flags; ship-to-ship transfers; falsified docs; 50-percent ownership rule; designations can land

- mid-voyage → periodic re-screening. The frontend retains an OpenSanctions-capable `intel/` server — hooks in naturally (stretch T-096). *Sources: NatLawReview, Alessa.*
6. **Emissions accounting** — GLEC v3.2 / ISO 14083:2023 WTW defaults: air freighter 0.608 kg CO₂e/tkm long-haul vs 1.516 short-haul (2.5×), sea container ~~0.008–0.016~~, road ~~artic 0.074–0.107~~, rail ~~0.011 (EU electric)–0.031 (diesel)~~, hub handling ~~+3–7 kg/TEU~~. ~~Implies leg-length-aware air factors and a carbon price line (\$80/t, EU ETS maritime 100% phase-in from 2026).~~ *Sources: GreenCalculus GLEC, factor dataset.*
7. **Disruption playbooks** — the industry runs named scenario playbooks: trigger event + pre-priced response ("Red Sea incident → rail/air for critical SKUs, tolerate longer ocean for the rest"; "US East Coast strike ≥2 weeks → divert West Coast + intermodal"), pre-approved alternates with dollar triggers, and **decision latency (signal→decision) as a KPI**. *Sources: GoFreight, CXTMS, AMB Logistic.*

10.2 The objects (all versioned, provenance=CALIBRATED, mandatory source field)

Object	Seeded with	Consumed by
<code>policy_rules</code>	SLA penalty rates per customer segment + 10% cap + grace days; severity \$ bands; trust-gate severities; carbon price \$80/t	<code>financial_engine</code> · <code>alert_engine</code> · <code>trust gate</code> · copilot context
<code>demurrage_tariffs</code>	Per-port <code>free_days</code> + tiered rate cards (replaces the flat <code>demurrage_rate_usd</code>)	<code>financial_engine</code> demurrage calc

escalation_matrix	Action → min role → auto-approve limit (OPERATOR ≤\$5k / MANAGER ≤\$25k / DIRECTOR >\$25k); ack-SLA minutes per severity (CRIT 15 min)	decision approval endpoint · sla_monitor
playbooks	Disruption type → allowed option types (DIVERT/ MODAL_SHIFT/SPLIT/ ACCEPT), max cost multiplier (≥1.5× needs escalation — the 15–30% off-guide finding), max ETA slip, force_majeure flag	reroute_engine filtering
emission_factors	GLEC v3.2 table by mode × haul-length, hub defaults	route_planner CO ₂ · ESG report · option scoring
compliance check (stretch)	Vessel IMO × OpenSanctions; AIS-gap red flags; new disruption type COMPLIANCE_HOLD	T-096 · vessel assignment script 06

Resolution & traceability: `services/policy_engine.py` resolves any rule via `resolve(key, context)` walking GLOBAL → customer-segment → port → lane specificity (60 s cache), and stamps a `policy_version` recorded on every alert, disruption, and decision row in the immutable audit log. Changing any policy value requires the DIRECTOR role and auto-bumps the version (which is itself a policy rule). Review cadence: quarterly, or when a tariff/regulation changes. **Member 4 is the fact-checker of record for these numbers** (decision D-Q8).

11. User Roles & the Application Surface

11.1 Application roles (authorization)

Role	Can do	Approval authority
OPERATOR	Monitor map, inspect shipments, acknowledge alerts, confirm/dismiss intel candidates, approve decisions	≤ \$5,000 total option impact
MANAGER	Everything above	≤ \$25,000
DIRECTOR	Everything above + change policy values (bumps <code>policy_version</code>)	> \$25,000

Approval enforcement (T-095): `POST /approve` validates the chosen option's `total_financial_impact` against the caller's limit → **403 naming the required role** when exceeded; the options modal shows the approver's remaining authority and disables cards above it.

11.2 Seeded demo users

Role	Email	Password
Operator	<code>operator@nexafreight.com</code>	<code>OperatorPass123!</code>
Manager	<code>manager@nexafreight.com</code>	<code>ManagerPass123!</code>
Director	<code>director@nexafreight.com</code>	<code>DirectorPass123!</code>

(All rotated before v2.0 ships — T-072.)

11.3 Pages & surfaces (end state)

- **The globe (home):** all layers, raw OSINT context + backend impact rings, live assets with provenance pills, feed-health HUD.
- **ShipmentInspectorPanel:** opens from route/asset/port clicks —

shipment summary, SLA deadline vs P50, delay-risk badge (green/amber/red), P10–P85 ETA range bar, top-5 SHAP bars, model version + provenance footer, demand-forecast tab, financials snapshot, copilot thread.

- **AlertCenter**: severity-sorted alert drawer; card = type icon, shipment ref, +N days ETA impact, \$ exposure, age; expand → per-order SLA breach table; acknowledge; manual disruption form; candidate confirm/dismiss queue.
 - **RerouteOptions modal**: disruption summary → per-order breach table → 3 option cards (ETA, cost Δ , breaches, CO₂ Δ , total impact; ☆ AI pick) → "cannot be undone" confirm → map redraw.
 - **/audit**: filterable, expandable decision history (all 3 presented options vs chosen), CSV export, read-only.
 - **/shipments/[id]**: full detail — leg timeline (planned vs actual), per-order SLA status, event log, route-version history ("v2 applied · approved by ..."), ML panel, financials, copilot.
 - **/analytics**: KPI cards, date filter, financial scorecard, CO₂ by mode vs air baseline, SLA heatmap (mode × region), demand charts — all DuckDB, < 1 s.
 - **Copilot**: Q&A thread with model badge (Gemini/Ollama), expandable "data used" section, 15 s timeout, honest 503. Answers only from injected context (shipment state, alerts, prediction, financials, applicable policy rules, matched intel events), never invents figures, cannot approve.
-

12. Team Structure — the 4-Role Model

The original plan used 6 roles; the team runs **3 engineering roles + 1 research member**. Every task carries exactly one owner and one difficulty tier (EASY / MED / HARD).

ML-ROUTEOPT — Machine Learning & Route Optimization

Model artifacts, feature contracts, SHAP, scoring functions, spatial/

geometry algorithms. Owns: T-040 prediction internals, the geo-matcher (Shapely buffers, polyline n, chokepoints), reroute scoring & SPLIT economics, copilot context engineering, v2.0.0 retrain (stretch). **Load: 5 core + 2 stretch · holds 2 of the 5 HARD tasks.**

DATA·DEVOPS — Data Engineering & DevOps

Migrations, ingest adapters, datasets, DuckDB analytics SQL, CI/CD, environments, secrets, indexes, hardening. Owns: repo restructure + drift merge, intel feed ingestion, corridor seeds, policy registry schema + seeds, analytics + hardening. **Load: 14 core + 1 stretch · bookends the timeline (Phase R & Phase 8).**

FE·BE·FS — Frontend / Backend / Full-Stack

FastAPI routers & service wiring, workers, business-logic services, all React/Next.js UI, SSE, integration tests. Owns: alert/financial/disruption/decision services + APIs, panels/drawers/modals/pages, LLM adapters, policy wiring + approval enforcement, demo staging. **Load: 31 core + 1 stretch · 53% of the plan — the volume role, staff accordingly (decision D-Q7).**

RESEARCH — Member 4 (paper & knowledge, non-coding)

Venue scouting, literature survey, competitive matrix, dataset/model cards, methodology, evaluation design, figures, full paper draft, and **fact-checker of record for policy numbers**. No coding; anything executable is run for them by engineering (standing 30-min weekly ritual). **Load: 8 core + 1 stretch · W1–W11 continuous.**

Workload matrix (58 core)

Phase	ML·RO	D·DO	FBE	RES	Σ
R · Reconciliation	—	4	2	—	6

4 · Predictions UI	2	1	2	—	5
5 · Intel + Alerts + Policy	1 (HARD)	3	12	—	16
6 · Rerouting	1 (HARD)	1	7	—	9
7 · Copilot	1	—	4	—	5
8 · Analytics + Polish	—	5	4	—	9
P · Paper (parallel)	—	—	—	8	8
Total core	5	14	31	8	58
Stretch (post-v2)	2	1	1	1	5

Difficulty × role: ML·ROUTEOPT 1E/2M/2H · DATA·DEVOPS 5E/8M/1H · FE·BE·FS 7E/22M/2H · RESEARCH 4E/4M/0H · **Total 17 EASY / 36 MED / 5 HARD.**

13. The Roadmap, Phase by Phase

Execution model: phases run in strict order **R → 4 → 5 → 6 → 7 → 8**; inside a phase, tasks parallelize across roles. Phase P (paper) runs continuously in parallel. Dates assume a **Monday 7 Sep 2026** start.

Each phase below ends with **"What to expect at the end of this phase"** — the observable outcome you can verify yourself, derived from the phase's Definition-of-Done.

PHASE R — Reconciliation & Baseline · *Week 1 (7–13 Sep) · 6 tasks*

Goal: fix the five drift issues (D1–D5) so every later phase builds on a clean skeleton. **Blocks everything — no new migration may land before the Alembic chain is merged.**

ID	Task (essentials)	Owner	Diff
T-073	Repo restructure: <code>backend/</code> + <code>frontend/</code> + <code>docs/</code> layout, archive the throwaway Vite app, externalize the 244 MB datasets, root README skeleton	D·DO	EASY
T-074	Backend drift merge (fixes D1): port <code>event.py</code> , <code>reroute_option.py</code> , migrations 003/004, richer <code>auth.py</code> from old HEAD — keep new waterway routing, looping sims, map router; regenerate dev DB; pytest green	D·DO	MED
T-075	Dead-UI prune (fixes D4): remove data-less layer toggles, quakes filter, unrendered ScmPanel, RECON panel (or document); decide <code>intel/</code> server fate (recommend keep + document)	FBE	EASY
T-076	Commit the unpushed ML client functions (fixes D2) + tests; <code>UNKNOWN_LANE 404</code> → null verified	FBE	EASY
T-077	Unified DevEx: root Makefile (<code>make dev-backend/dev-frontend/test/...</code>), one runbook (ports, creds, one-command seed), dual GitHub Actions CI (backend: <code>ruff+mypy+pytest</code> · frontend: <code>tsc+vitest+next</code> build)	D·DO	MED
T-078	Secrets & env alignment: reconcile <code>.env.examples</code> (keep proxy, SSE direct), gitleaks sweep = 0, rotate demo creds	D·DO	EASY

✓ **What to expect at the end of Phase R (M0 · 13 Sep):** a clean two-folder repo any team member can clone and run with `make dev-backend+make dev-frontend`; `alembic upgrade head` → `seed` → green `pytest` in one script; all 431 Vitest tests + `tsc` + `next build` green after the move; zero dead toggles on the layer panel; no secrets in git history. Nobody writes a new feature until this is green.

PHASE 4 — Predictions in the Control Tower · *Weeks 2–3 (14–27 Sep) · 5 tasks*

Goal: unblock every ML-visible UI by building the one endpoint everything needs, then showing predictions honestly in the inspector.

ID	Task (essentials)	Owner	Diff
T-040	[THE BLOCKER] GET /api/shipments/{id}/predict — server-side feature assembly → delay probability, <code>is_high_risk</code> , top-5 SHAP factors (feature, impact, direction, plain English), P10/P50/P85 ETAs, <code>sla_risk</code> band, <code>model_version</code> , <code>provenance=DERIVED</code> ; 503 when models unloaded	ML·RO	MED
T-079	ShipmentInspectorPanel v2 — real side panel (not popup) from entity clicks: summary, SLA vs P50, risk badge, P10–P85 range bar, SHAP bars, provenance footer	FBE	MED
T-043	GET /api/analytics/demand via DuckDB reading SQLite directly; chart-ready p50/lower/upper series	D·DO	MED
T-080	Demand-forecast tab: 30/60/90-day sparkline; the 596 UNKNOWN_LANE lanes get a calm "insufficient history" empty state	FBE	EASY
T-042	ML inference unit tests: fixture extraction, response schema lock, SHAP sum sanity, $P10 \leq P50 \leq P85$ invariant, provenance on every prediction — <10 s in CI	ML·RO	EASY

✓ **What to expect at the end of Phase 4 (M1 · 27 Sep):** clicking any moving ship/truck/plane opens a panel showing its delay-risk badge, an honest ETA *range*, and the top-5 reasons *why* (in plain English) — all from one API call. The demand tab either shows a forecast or says

"insufficient history," never a broken chart. All three test suites green.

PHASE 5 — Disruption Intelligence + Alerts & Financial Engine + Policy Layer · Weeks 4–6 (28 Sep–18 Oct) · 16 tasks · CORE NEW SCOPE

Goal: the reason this integration exists. Build the intel pipeline (news → matched impact → alert), the telemetry detectors, the financial engine, and the researched policy layer underneath all of it.

ID	Task (essentials)	Owner	Diff
T-093	[NEW] Policy & rules schema + seeds (migration 006): <code>policy_rules</code> , <code>demurrage_tariffs</code> , <code>escalation_matrix</code> , <code>playbooks</code> , <code>emission_factors</code> with the researched defaults (§10) + docs/ <code>policy_defaults.md</code> with sources cited	D·DO	MED
T-081	[NEW] <code>intel_events</code> schema (migration 005) + GDELT adapter (GEO 2.0 + DOC themes, 15-min cadence, keyless)	D·DO	MED
T-082	[NEW] GDACS + EONET adapters (cyclones/floods/quakes/volcanoes with severity; storms/wildfires), normalized into <code>intel_events</code> , recorded fixtures	D·DO	MED
T-083	[NEW · HARD] <code>services/intel_matcher.py</code> — the heart of the integration: port-in-radius, route-polyline n chokepoint buffer, affected shipments, dedup per (event, entity, day), confidence attached	ML·RO	HARD

T-084	[NEW] <code>workers/intel_monitor.py</code> — GDELT 15 min / GDACS+EONET hourly; trust gate (HIGH+ → CANDIDATE for one-click confirm; severities from policy); kill-switch <code>INTEL_AUTO_DISRUPT=false</code> for demos	FBE	MED
T-046	Financial engine + <code>sla_checker</code> , industry-realistic: revenue × segment rate × days late past grace, capped ~10% , force-majeure exemption for FM-flagged disruptions, tiered demurrage from per-port tariff cards , carbon at policy price — all via policy engine, never hardcoded	FBE	MED
T-094	[NEW] <code>services/policy_engine.py</code> — scoped resolution + 60 s cache + <code>policy_version</code> stamping; refactors T-046/T-047/T-084 to read through it, regression-tested unchanged	FBE	MED
T-044	AIS deviation detector (> 10% planned-vs-actual → <code>VESSEL_DELAY</code> , idempotent)	FBE	MED
T-045	Port congestion monitor (daily 06:00; index > 1.4 → <code>PORT_CONGESTION</code>)	FBE	EASY
T-047	Alert engine: disruption → revised ETA → per-order SLA check → \$ exposure → severity from policy bands; ack-SLA auto-escalation; one alert per (shipment, disruption)	FBE	MED
T-048	<code>sla_monitor</code> worker — 15-min scan, creates/resolves <code>SLA_RISK</code> alerts	FBE	MED
T-049	Real alerts/disruptions APIs + new <code>GET /api/intel/events</code> (map payload), <code>GET /api/intel/events/{id}</code> (affected shipments server-side), candidate confirm/dismiss endpoints	FBE	MED

T-085	[NEW · HARD] "Logistics Impact" map layer — impact rings + matched-lane highlight pulses, visually distinct from raw OSINT dots; intel inspector; LayerPanel entries. Deep surgery in a 3.8k-line component — senior-led, pair review	FBE	HARD
T-050	The AlertCenter: severity-sorted drawer, breach tables, acknowledge, manual disruption form, candidate queue	FBE	MED
T-051	Unit tests: financial/sla known values; matcher geo tests (perimeter, chokepoints, dedup, confidence bands)	FBE	MED
T-052	Integration test: synthetic GDELT fixture → candidate → confirm → alert with exposure → acknowledge → audit entry	FBE	MED

✓ **What to expect at the end of Phase 5 (M2 · 18 Oct — the headline moment):** inject a synthetic "conflict spike near Bab-el-Mandeb" fixture and, within one poll cycle: an impact ring appears on the globe, Red Sea lanes highlight, a candidate disruption appears, the operator confirms it, and an alert shows the correct dollar exposure and per-order breach table — every figure traceable to a versioned policy rule (capped SLA penalty, tiered demurrage, FM exemption all exercised in tests). A manually-posted disruption becomes an alert in under 30 seconds.

PHASE 6 — Rerouting & Decisions · Weeks 7–8 (19 Oct–1 Nov) · 9 tasks

Goal: close the loop — from "here is the problem and what it costs" to "here are your options," with role-gated approval and an immutable audit trail.

ID	Task (essentials)	Owner	Diff
----	-------------------	-------	------

T-053	Seed corridor alternatives as data, now including geopolitical detours the intel layer exposes (Red Sea → Cape of Good Hope +10–14 d; Suez → overland/air; canal-closure alternates) + when_to_use rules	D·DO	MED
T-054	[HARD] reroute_engine: ACCEPT_DELAY baseline always present + corridor-filtered alternates + SPLIT evaluation; full financials per option; ranked by total impact; ☆ is_ai_recommended; top 3 persisted; legality/cost/ETA guardrails from playbooks	ML·RO	HARD
T-055	[HARD] decision_executor: approve → REPLACE (never delete) legs, route_version++, SPLIT child shipment, immutable audit entry, P&L recalc, idempotent re-approval guard. Pair-reviewed state machine	FBE	HARD
T-056	Decision APIs: GET /api/alerts/{id}/options (lazy-generated), POST /approve, GET /api/decisions[/id] audit history	FBE	MED
T-095	[NEW] Approval-authority enforcement: option cost vs caller's matrix limit → 403 naming required role; modal shows remaining authority + disables over-limit cards; audit records policy_version + applied-rules snapshot; CRIT auto-escalation verified end-to-end	FBE	MED
T-057	RerouteOptions modal: 3 scored cards (ETA, cost Δ, breaches, CO ₂ Δ, total impact, ☆ AI pick) → confirm → map redraws, replaced legs greyed	FBE	MED

T-058	/audit page: filterable decision history, all options vs chosen, CSV export, read-only	FBE	EASY
T-059	Unit tests: baseline-always-present, corridor filter, SPLIT logic, AI-pick rule, REPLACE-not-delete, version bump, idempotency 400	FBE	MED
T-060	The definitive integration test: disruption → alert → options → approve → new GeoJSON → audit → re-approve 400	FBE	MED

✓ **What to expect at the end of Phase 6 (M3 · 1 Nov):** a Red-Sea intel disruption yields three scored options including a Cape-of-Good-Hope divert; approving one regenerates legs (`route_version=2`), the globe redraws the new path, and an immutable audit row records who approved what under which policy version. SPLIT appears exactly when delay penalties exceed the air premium. An OPERATOR attempting a \$40k modal shift gets a 403 naming the required MANAGER role.

PHASE 7 — Copilot & AI Narratives · Week 9 (2–8 Nov) · 5 tasks · *deliberately small*

Goal: add the AI assistant — grounded, auditable, offline-capable — without letting it eat earlier phases. (First safe-to-cut phase if anything slips.)

ID	Task (essentials)	Owner	Diff
T-061	LLM adapters (Gemini Flash free tier + Ollama llama3.1:8b offline) behind one protocol; 429 → Ollama fallback; every call hash-logged; zero DB write access	FBE	MED

T-062	Copilot service + POST /api/copilot/ask: context = shipment state + alerts + ML prediction + financials + applicable policy rules (fulfills the plan's promised APPLICABLE RULES block) + matched intel events. Persona: answer only from context, never invent figures, cannot approve	ML·RO	MED
T-063	Copilot panel in the inspector: Q&A thread, model badge, "data used" expander, 15 s timeout, honest 503	FBE	EASY
T-088	Move the frontend's client-side Gemini logistics summaries behind a backend POST /api/copilot/narrate (context-scoped, audited); pure-OSINT summaries may stay client-side — recorded in docs	FBE	MED
T-064	Smoke tests with MockLLM: risk answer cites ML data, cost answer cites financials, out-of-context → graceful refusal, audit entry per call, 403 on foreign shipment	FBE	EASY

✓ **What to expect at the end of Phase 7 (M4 · 8 Nov):** asking *"Why is shipment NF-2024-XXXXXX at risk?"* answers with its delay probability, its P85, and the actual matched news event — citing the policy rule behind any dollar figure. Fully functional offline on Ollama. Every LLM call appears in the audit log; no logistics summary is generated in the browser anymore.

PHASE 8 — Analytics, Hardening & Demo · Weeks 10–11 (9–22 Nov) · 9 tasks · v2.0 ship

ID	Task (essentials)	Owner	Diff
----	-------------------	-------	------

T-065	DuckDB analytics suite: financial scorecard, ESG (CO ₂ by mode vs air baseline), SLA heatmap (mode × region), P&L breakdown — all < 1 s	D·DO	MED
T-066	<code>/analytics</code> route: KPI cards, date filter, stacked CO ₂ bars, SLA heatmap, demand charts (recharts)	FBE	MED
T-067	Index tuning; list endpoints < 200 ms @ 10k shipments	D·DO	EASY
T-068	Feed-health HUD extended to intel adapters; GET <code>/api/shipments/{id}/financials</code> P&L snapshot in the inspector	FBE	MED
T-069	<code>/shipments/[id]</code> full detail: leg timeline planned-vs-actual, per-order SLA status, event log, route-version history, ML panel, financials, copilot	FBE	MED
T-089	[HARD] Production hardening: clean <code>next build</code> + prod smoke; SSE proxy fix; refresh-token flow (page reload no longer logs out); marker-performance budget; one-command <code>seed_demo.py</code> reproducing the 600-leg demo DB	D·DO	HARD
T-070	Demo script v2 + <code>seed_demo_data.py</code> (see §18 for the 10-minute narrative)	FBE	MED
T-071	README: what it is, mermaid architecture, setup from clone, data sources, screenshots, provenance rule (reuses T-099's competitive matrix)	D·DO	EASY
T-072	Security audit: gitleaks both repos (0), rotate keys + demo creds, bandit, explicit CORS, dependency audits, confirm <code>@google/generative-ai</code> removal	D·DO	EASY

✓ What to expect at the end of Phase 8 (M5 · 22 Nov — v2.0 ships):

analytics render real figures in under a second; the 10-minute demo runs from fixtures with zero live-feed dependency; a stranger can run the entire system from the README alone; `make check` (pytest + vitest + tsc + lint) is green; every known secret rotated.

PHASE P — Research & Conference Paper (parallel track, Member 4) · Weeks 1–11 continuous · 8 tasks · NON-CODING

Engineering's only obligation: 30 min/week of fact-checks + running the measurements M4 designs.

Candidate paper framing (finalized in T-101): *"NexaFreight: provenance-aware multimodal freight monitoring with OSINT-driven disruption intelligence"* — four contributions: **(1)** one provenance discipline unifying REAL / REPLAYED / SIMULATED / DERIVED / CALIBRATED telemetry; **(2)** geo-correlation of public event streams (GDELT · GDACS · NASA EONET) to shipments, ports, and sea lanes; **(3)** SHAP-explained LightGBM delay + quantile ETA prediction on 180k real orders; **(4)** every operator decision recorded against a versioned policy layer for auditability. Target: IEEE-format applied conference (6–10 pp); venue picked by T-097 (decision D-Q9); **arXiv preprint by 22 Nov is the guaranteed floor.**

ID	Task (essentials)	Weeks	Diff
T-097	Venue & deadline scan (5–8 venues: deadlines, page limits, fees, review model) + Zotero + <code>references.bib</code> + IEEE template + <code>paper/</code> folder → <code>venue_matrix.md</code>	W1–2	EASY
T-098	Related-work survey: 20–30 sources in 4 clusters (commercial/academic control towers, AIS-based ETA prediction, GDELT geo-analysis, disruption management) + novelty-gap table — the paper's spine	W2–4	MED

T-099	Competitive matrix: project44 · FourKites · Flexport · SAP TM · Kinaxis vs NexaFreight (real AIS, OSINT fusion, open-source, \$0 infra, policy-governed decisions, explainable ML) — reused in the README	W2–3	EASY
T-100	Dataset & model cards (DataCo, UN/LOCODE, IMF, feeds; AUC 0.731 vs 0.707, ETA pinball/coverage, demand WAPE, 79/675 lanes) + provenance appendix	W3–5	EASY
T-101	Methodology section: architecture narrative, provenance discipline formalized, <code>intel_matcher</code> pseudocode, policy-layer design (engineering fact-checks)	W4–7	MED
T-102	Evaluation protocol + results on frozen fixtures : ML tables from <code>metadata.json</code> , matcher precision/recall on hand-labeled synthetic events, alert latency, decision-latency KPI	W7–9	MED
T-103	Figures & tables: architecture diagram, globe screenshots (rings + provenance pills), SHAP figure, decision modal, policy excerpt, Red-Sea scenario timeline	W8–10	EASY
T-104	Full draft (abstract→limitations) → all-hands internal review → venue submission or arXiv (cs.CY/cs.AI) preprint + slides skeleton	W9–11	MED

✓ **What to expect at the end of Phase P (by 22 Nov)**: a complete venue-format draft with ≥ 20 references, every result number reproducible from `metadata.json` or a frozen-fixture evaluation, ≥ 5 figures, internal review closed, and the paper **submitted to a venue or live on arXiv**. Stretch T-105 (rebuttals, camera-ready, poster, talk deck, repo tag `v2.0-paper`) covers the lifecycle after submission.

14. Milestones & Timeline

Milestone	Date	What you can see
M0	13 Sep	Clean baseline — Phase R merged, CI green, drift gone
M1	27 Sep	Predictions visible — risk/ETA/SHAP in the inspector
M2	18 Oct	News moves freight — intel → disruption → alert demo (headline moment)
M3	1 Nov	Operator in control — 3-option reroute approved, map redraws, audit immutable
M4	8 Nov	Copilot online — intel-aware answers, offline-capable
M5	22 Nov	v2.0 — analytics + hardened build + 10-min demo + README + audit

Progress honestly stated (6 Sep): original-plan progress ≈ 47% (~45 task-equivalents done incl. integration) · backend capability ≈ 62% (schema→ML done; alerts/reroute/copilot/analytics ahead) · the OSINT↔logistics bridge ≈ 10% (context layers only).

Fastest demo-hungry path (if a checkpoint is needed sooner): Phase R → T-040 + T-079 → Phase 5 core slice only (T-081/083/084/049 + T-085 + T-050) = an intel-driven alert on the globe in ~5 weeks, deferring rerouting/copilot.

15. Stretch Backlog (post-v2.0)

ID	Idea	Owner	Diff
----	------	-------	------

T-090	ML schema v2.0.0 retrain — activate reserved news_risk_score + active_disruption_near_dest once intel history accumulates; validate AUC lift over 0.731	ML·RO	HARD
T-091	LLM triage of raw GDELT DOC articles into structured disruption candidates (replaces v1 keyword rules)	ML·RO	HARD
T-092	Lane risk heat layer (30-day rolling intel-event density) + customer-facing delay comms drafts + webhook notifications	FBE	MED
T-096	Compliance screening — vessel IMO × OpenSanctions at assignment + nightly re-screen; AIS-gap red flags; COMPLIANCE_HOLD disruption type with forced manual review	D·DO	MED
T-105	Paper lifecycle: rebuttals, camera-ready, poster + 10-min talk deck, preprint + artifact link (repo tag v2.0-paper)	RES	MED

16. Open Decisions

ID	Question	Recommendation
D-Q1	Two repos vs the pushed single combined repo?	Embrace the monorepo layout (backend/ + frontend/), keep the two-process rule — the "separate repos" intent was about code boundaries, which stay intact

D-Q2	Auto-create disruptions from news, or human-in-the-loop?	Hybrid trust gate: LOW/MED auto, HIGH+ candidate-confirm; demo mode sets <code>INTEL_AUTO_DISRUPT=false</code> with replay fixtures
D-Q3	Does OSINT news get its own provenance label?	No — keep the five labels. Event = REAL, match = DERIVED, candidate status surfaced separately. A sixth badge would weaken the rule
D-Q4	Charting library?	Add <code>recharts</code> (React-19 compatible); <code>lightweight-charts</code> stays removed
D-Q5	Keep the <code>intel/</code> Express KG server and RECON panel?	Keep + document <code>intel/</code> (the knowledge graph is a differentiator); drop or feature-flag RECON (off-scope, needs an external scanner)
D-Q6	Refresh tokens?	Yes — T-089. In-memory-only JWT logs out on every reload; acceptable pre-demo, not for v2.0
D-Q7	Is FE·BE·FS one person or several?	It carries 31/58 core tasks — the critical path. If one person: protect their W4–W8 completely, treat T-085 as two-week, DATA·DEVOPS absorbs endpoint work permanently. Decide by end of W1
D-Q8	Who owns the policy numbers?	Researched defaults seeded as <code>CALIBRATED</code> + mandatory <code>source</code> ; changes require DIRECTOR and auto-bump <code>policy_version</code> ; quarterly review; Member 4 = fact-checker of record
D-Q9	Paper venue?	T-097's scan decides in W2. Aim at an applied IEEE-format conference (deadline Dec–Feb); arXiv preprint by 22 Nov is the guaranteed floor ; submit to the institute conference too if one exists

17. Risk Register

Risk	Severity	Mitigation
Drift re-emerges between backends	HIGH	After T-074: new repo's <code>backend/</code> is the only writable backend; <code>Winter_Soldiers</code> archived read-only with a pointer README
News false positives create nonsense alerts	HIGH	Trust gate, conservative radii, dedup per (event, entity, day), confidence stored + displayed; demo never calls live feeds
Role imbalance — 53% of tasks on FE·BE·FS	HIGH	Resolve D-Q7 in W1; pair all 5 HARD tasks; DATA·DEVOPS permanently owns SQL endpoints; ML·ROUTEOPT idle windows feed stretch + design + paper fact-checking
Phase 5 scope creep eats rerouting/copilot time	MED	The fastest-demo path defines the minimum viable Phase 5 slice; everything else is stretch; Phase 7 is the first safe cut
SSE buffering behind Next rewrites in prod	MED	Known gotcha: EventSource connects direct to :8000 (CORS allowlist) — codified in T-089
GDELT/GDACS rate or format changes	MED	Adapters behind Protocol + recorded fixtures; demo uses replay
596/675 demand lanes 404 looks like breakage	LOW	Calm "insufficient history" empty state + explainer copy
Browser perf with 36+ animated markers + rings	LOW	Performance budget in T-089; symbol layer/cluster past ~100 concurrent

Seeded policy values drift from reality	LOW	Versioned + sourced policies; audit pins <code>policy_version</code> ; quarterly review
Paper results blocked by build slippage	MED	Frozen-fixture evals give stable numbers by W9 regardless of UI polish; arXiv floor; M4's track never loads the engineering critical path
Research member blocked waiting on engineering	LOW	Standing 30-min weekly ritual; figures reuse demo assets; M4 front-loads T-097/T-098/T-099/T-100 which need zero engineering output

18. What to Expect at the Very End

18.1 The product you will have on 22 Nov 2026

A running two-process system where a **DIRECTOR can sit down and, in ten minutes:**

1. Show the globe with **36 live assets** gliding along 600 waterway-aware routes, provenance pills on everything, feed-health HUD green.
2. Click a vessel → inspector shows SLA deadline vs prediction, a red/amber/green delay-risk badge, a **P10–P85 ETA range**, and the **top-5 SHAP reasons in plain English**.
3. At minute 3, a **pre-staged Red-Sea intel event fires** (deterministic fixture) → impact ring blooms on the map, matched lanes pulse, a candidate disruption appears → one click confirms it → an **ALERT with real dollar exposure** (tiered demurrage, capped SLA penalty, policy-version stamped).
4. Open the alert → **3 scored options**: accept the delay, divert via Cape of Good Hope, split to air — with cost Δ, CO₂ Δ, breach counts, and an ☆ AI recommendation.
5. Approve the Cape divert → "cannot be undone" confirm → **legs**

regenerate, the map redraws the new path, replaced legs greyed.

6. Ask the copilot *"why was this rerouted?"* → it answers citing the news event, the prediction, and the exact policy rule that priced it — **offline, on Ollama, if needed**.
7. Open `/audit` → the immutable record of the decision, all three options vs the chosen one, `policy_version` pinned. Open `/analytics` → financial scorecard, CO₂ vs air baseline, SLA heatmap.
8. And every number on every screen answers the question *"is this real?"* — REAL, REPLAYED, DERIVED, CALIBRATED, or SIMULATED — visibly.

18.2 The artifacts

- **The application** (v2.0): hardened, refresh-token auth, one-command demo seed, `make check` green, README a stranger can follow.
- **The paper**: IEEE-format 6–10 pp, ≥20 references, ≥5 figures, frozen-fixture reproducible results — submitted to a venue **or live on arXiv** by 22 Nov, with a slides skeleton. Post-v2 stretch covers rebuttals, camera-ready, poster, and the talk deck.
- **The policy pack**: `docs/policy_defaults.md` — every industry rule the system encodes, with sources, versioned in the DB.
- **The knowledge base**: dataset/model cards, competitive matrix, related-work survey, venue matrix — all reusable beyond this project.

18.3 Honest caveats

- The seeded policy numbers are **research-informed defaults (CALIBRATED)**, not a legal/commercial source of truth — they exist to make the system's mechanics real and are versioned so they can be swapped.
- News matching v1 is rule-based geometry with a human trust gate, by design — it will miss subtle events and occasionally produce candidates that operators dismiss. That humility is a

feature's selling point, not a bug (and T-090/T-091 are the planned upgrades).

- The system monitors and recommends; it does not contact carriers, file claims, or execute bookings.
-

19. Appendix — Standing Conventions, Credentials, Glossary

Carried-forward conventions (treat as law)

- GeoJSON is always `[lon, lat]` · timestamps ISO-8601 UTC · every position/prediction carries **provenance** and gets visibly badged · JWT in memory only (until T-089) · **SSE connects direct, not through Next rewrites** · ETAs are P10–P85 ranges, never single dates · `UNKNOWN_LANE 404` → calm empty state · restart the Next dev server after `.env.local` edits · **The frontend never imports Python, never reads the DB, never duplicates business logic.**

Ports & processes

- Backend: FastAPI on **:8000** · Frontend: Next.js on **:3000** · Frontend proxies `/api/nexa/*` → :8000 (SSE excepted, direct) · optional `intel/` Express server for the knowledge graph.

Glossary

- **Control tower** — a system that watches all shipments, detects problems, prices them, and supports human decisions.
- **Provenance labels** — REAL (live external) · REPLAYED (recorded real) · DERIVED (computed from real) · CALIBRATED (industry-informed default, sourced) · SIMULATED (synthetic).
- **Trust gate** — rule that HIGH-severity news-derived disruptions

enter as CANDIDATES needing one-click human confirmation.

- **Policy version** — monotonically increasing stamp recorded on every alert/decision; changing any policy value (DIRECTOR-only) bumps it.
 - **Chokepoint registry** — reference data: Bab-el-Mandeb, Hormuz, Malacca, Suez, Panama.
 - **SPLIT** — decision option that divides a shipment into a child shipment on a faster mode (e.g. air) when delay penalties exceed the modal premium.
 - **M0–M5** — milestones (§14). **T-xxx** — task IDs, continuous with the original T-001–T-072 plan.
 - **MLE/DoD** — per-phase Definition-of-Done: the observable "what to expect" listed at the end of each phase in §13.
-

Report ends. Companion documents:

nexafreight_scheduled_flow_v2.html (task schedule with workload matrices and week-by-week allocation) · source PDFs NexaFreight.pdf, scheduled_flow.pdf, and the project-state memo (in uploads/).